

Het integreren van Python modules in een .NET rekenmodel.

Een proof-of-concept bij ILVO.

Roetynck Caradoc.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. J. Willem

Co-promotor: Dhr. S. Bosch

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Tijdens een gesprek met ILVO kwam de vraag naar voren of het mogelijk was om een probleem op te lossen dat zij hadden. Het onderwerp kwam op omdat hun onderzoekers hun berekeningsmodellen willen testen en aanpassen, maar ze hebben geen ervaring genoeg met C# om dit zelf te doen. De onderzoekers hebben wel ervaring met Python en daarom leek het een goed idee om een Jupyter-notebook te maken waarin de onderzoekers kunnen experimenteren met de modellen.

Ik heb gebruik gemaakt van genAI voor de grammatica & zinsbouw te verbeteren. Veel dank aan mijn moeder voor het helpen bij het schrijven van deze bachelorproef, aan mijn copromoter, Samuel Bosch, en aan mijn promotor, Jan Willem, voor hun begeleiding en feedback tijdens het proces.

Samenvatting

Bij ILVO, het Instituut voor Landbouw-, Visserij- en Voedingsonderzoek, draaien veel onderzoeksprojecten rond complexe rekenmodellen. Deze modellen beschrijven hoe je emissies moet berekenen, of hoe bepaalde bedrijfsparameters zich verhouden tot een bepaald resultaat. Het probleem: onderzoekers schrijven deze modellen op, maar IT-developers implementeren ze vervolgens in C# en .NET. Zodra dat gebeurd is, zijn onderzoekers min of meer afhankelijk van de IT-afdeling voor elke kleine aanpassing of test. De code is als een zwarte doos. Ze zien hun formule niet terug, ze begrijpen niet precies wat er ondertussen gebeurt, en elke keer dat ze iets willen proberen, moet de IT-afdeling er aan te pas komen.

Deze bachelorproef bestudeert hoe die kloof kan worden gedicht. Het gaat uit van documentatie bij ILVO, analyseert waar de wrijving zit, en beschouwt welke bestaande werkwijzen uit soortgelijke domeinen (literate programming, Jupyter notebooks, documentatietools) daar nuttig voor kunnen zijn. Met die inzichten is een proof-of-concept gebouwd als proof-point: kunnen onderzoekers meer autonomie krijgen en meer inzicht in hoe hun model werkt, zonder dat de IT-kant eronder lijdt?

Het blijkt inderdaad mogelijk. Door beter inzicht in de scheiding tussen wat onderzoekers moeten begrijpen en wat de implementatie doet, kan zo'n samenwerking veel vloeiender worden ingericht. Er is niet alles opnieuw nodig, maar wel betere documentatie, betere structuur, en ruimte voor onderzoekers om voorzichtig te experimenteren.

Inhoudsopgave

Lijst van figuren	vi
Lijst van tabellen	vii
Lijst van codefragmenten	viii
1 Inleiding	1
2 Stand van zaken	4
3 Methodologie	5
4 Probleem Stelling	6
4.1 Huidige situatie	6
4.2 Doel	7
5 Conclusie	8
A Onderzoeksvoorstel	10
A.1 Inleiding	10
A.2 Literatuurstudie	12
A.2.1 Literate programming	12
A.2.2 Org-mode Babel	12
A.2.3 Jupyter notebook	13
A.2.4 RMarkdown	13
A.2.5 Tussenbesluit	14
A.3 Methodologie	14
A.3.1 Eerste fase	14
A.3.2 Tweede fase	15
A.3.3 Derde fase	15
A.3.4 Vierde fase	15
A.3.5 Vijfde fase	15
A.4 Verwacht resultaat, conclusie	16
A.4.1 Probleem- en domeinanalyse	16
A.4.2 Literatuurstudie & technologieverkenning	16
A.4.3 Proof-of-Concept ontwikkeling	16
A.4.4 Evaluatie en validatie	16
Bibliografie	17

Lijst van figuren

Lijst van tabellen

Lijst van codefragmenten

1

Inleiding

In onderzoeksprojecten worden berekeningen vaak eerst uitgewerkt door onderzoekers en pas daarna omgezet naar software. Dat is logisch: onderzoekers kennen het domein en de formules, terwijl ontwikkelaars instaan voor de technische uitwerking. Toch kan die verdeling ook voor problemen zorgen. Deze bachelorproef onderzoekt dat probleem binnen de context van ILVO en het EMAV-project. Bij ILVO (Instituut voor Landbouw-, Visserij- en Voedingsonderzoek) worden rekenmodellen gebruikt om onder andere milieu-impact en emissies binnen de landbouwsector te berekenen. Zulke modellen vertrekken vanuit wetenschappelijke kennis, formules en aannames die door onderzoekers worden opgesteld. Wanneer een model in een toepassing gebruikt moet worden, volstaat een Excel-bestand of los document meestal niet meer.

De berekeningen moeten dan worden omgezet naar software die betrouwbaar, onderhoudbaar en bruikbaar is voor eindgebruikers. In de onderzochte context gebeurt dat binnen een .NET-omgeving, met C# als belangrijkste programmeertaal.

De onderzoekers die met de modellen werken, zijn niet noodzakelijk vertrouwd met C#. Ze zijn vaak wel vertrouwd met de achterliggende formules, de parameters en de betekenis van de resultaten. Daardoor ontstaat een praktische vraag: hoe kan de software zo worden ingericht dat onderzoekers de berekeningen beter kunnen volgen en eventueel zelf kleine experimenten kunnen uitvoeren?

In de praktijk ontstaat er een afstand tussen de wetenschappelijke versie van het model en de technische implementatie ervan. Onderzoekers leveren formules, parameters of voorbeeldberekeningen aan. Ontwikkelaars vertalen die vervolgens naar C#-code, koppelen ze aan databanken en verwerken ze in de rest van de applicatie.

Dat werkt zolang het model stabiel blijft. Zodra een onderzoeker een formule wil aanpassen, een parameter wil testen of een alternatief scenario wil vergelijken, is

er opnieuw technische hulp nodig. De onderzoeker kan de berekening meestal niet rechtstreeks uitvoeren in de softwarecontext waarin het model echt gebruikt wordt. Omgekeerd is het voor ontwikkelaars niet altijd eenvoudig om de wetenschappelijke bedoeling achter elke berekeningsstap te blijven volgen.

De kern van het probleem is dat de berekeningslogica na verloop van tijd moeilijk zichtbaar wordt voor de onderzoeker. De code voert het model wel uit, maar de wetenschappelijke redenering zit verspreid over documentatie, broncode, overleg en voorbeeldbestanden. Daardoor kan de toepassing voor onderzoekers aanvoelen als een black box.

Een bijkomend probleem is dat documentatie snel achterloopt op de implementatie. Wanneer code wijzigt maar de uitleg niet mee aangepast wordt, ontstaat "documentation drift". Dat maakt het moeilijker om na te gaan of de software nog overeenkomt met het model zoals onderzoekers het bedoelen. Deze bachelorproef onderzoekt daarom hoe code, documentatie en experimenteerruimte dichter bij elkaar gebracht kunnen worden.

Deze bachelorproef probeert niet het volledige EMAN-project te herwerken. Ook het volledige emissiemodel zelf wordt niet opnieuw ontworpen. De scope ligt bij een proof-of-concept waarin wordt nagegaan hoe C#-logica beschikbaar gemaakt kan worden in een Python-omgeving en hoe een alternatieve implementatie getest kan worden zonder de hoofdapplicatie rechtstreeks aan te passen.

De nadruk ligt dus op de technische haalbaarheid en op de bruikbaarheid van de werkwijze voor onderzoekers. Beveiliging, performantie en integratie in een productieomgeving worden besproken waar ze nodig zijn om de PoC (proof-of-concept) te beoordelen, maar ze worden niet volledig uitgewerkt.

Vanuit deze problematiek is de volgende centrale onderzoeksvraag geformuleerd: *Hoe kunnen we de kloof tussen IT-logica en wetenschappelijke analyse verkleinen, zodat onderzoekers zonder diepgaande programmeerkennis inzicht krijgen in hun modellen en er zelf mee kunnen experimenteren?*

Om deze hoofdvraag te beantwoorden, worden de volgende deelvragen onderzocht:

1. Welke onderdelen van de bestaande code en architectuur maken het moeilijk voor onderzoekers om de berekeningen te volgen?
2. Waarom volstaat de huidige documentatie niet om het model en de implementatie samen duidelijk te maken?
3. Welke vormen van interactieve documentatie of Jupyter-notebooks zijn geschikt om berekeningen begrijpelijker te maken?
4. Hoe kan een onderzoeker veilig experimenteren met een alternatieve implementatie zonder de stabiliteit van de applicatie te schaden?

5. Hoe kan een proof-of-concept aantonen dat C#-logica en Python-code samen gebruikt kunnen worden binnen deze context?

Het doel van dit onderzoek is om een haalbare werkwijze te verkennen waarmee onderzoekers meer inzicht krijgen in de berekeningen, zonder dat de bestaande software volledig herbouwd moet worden. Daarvoor wordt een proof-of-concept uitgewerkt. De focus ligt niet alleen op documentatie, maar ook op de vraag hoe onderzoekers op een gecontroleerde manier met de berekeningen kunnen experimenteren.

De belangrijkste doelstellingen zijn:

- Architecturale scheiding: nagaan hoe domeinlogica binnen een .NET-project duidelijker gescheiden kan worden van infrastructuur zoals databanken en API's.
- Hybride integratie: onderzoeken hoe C#-modules via pythonnet beschikbaar gemaakt kunnen worden in een Python-omgeving.
- Experimenteerruimte: bekijken hoe een onderzoeker een alternatieve implementatie kan testen zonder de officiële toepassing aan te passen.
- Evaluatie: beoordelen in welke mate de PoC helpt om berekeningen begrijpelijker en beter testbaar te maken.

De rest van deze bachelorproef volgt de deelvragen. Eerst wordt in de stand van zaken bekeken welke bestaande technieken en werkwijzen relevant zijn, zoals Python.NET, computationele notebooks, literate programming en containerisatie. Daarna licht de methodologie toe hoe het probleem en de mogelijke oplossing onderzocht werden.

Vervolgens wordt de architectuur van het proof-of-concept besproken, met aandacht voor de databank, API en notebooks. Daarna volgen hoofdstukken over de huidige documentatie, de complexiteit van het project, veilig experimenteren en de verweving tussen code en documentatie. Ten slotte wordt de proof-of-concept besproken en wordt in de conclusie teruggekoppeld naar de onderzoeksvraag en deelvragen.

2

Stand van zaken

3

Methodologie

afspraken maken voor gespreken gepserk met samuel, onderzoeks, mentor schrijven + sturen naar onderzoekers

4

Probleem Stelling

4.1. Huidige situatie

Het Emissie Model Ammoniak Vlaanderen (EMAV) is het officiële Vlaamse model voor de berekening van ammoniakemissies uit de land- en tuinbouw. Het model werd ontwikkeld om op een uniforme en wetenschappelijke onderbouwde manier de ammoniakuitstoot van verschillende landbouwactiviteiten te berekenen. Hiervoor combineert EMAV gegevens over onder meer dieraantallen, stalsystemen, mestopslag, mestaanwending en beweiding met emissiefactoren om de totale ammoniakemissie te bepalen (Foqué, 2009). De berekende emissies vormen de basis voor de Vlaamse emissie-inventaris en worden gebruikt in modellen die de verspreiding en depositie van stikstof simuleren, waardoor EMAV een belangrijke rol speelt in het Vlaamse stikstofbeleid en milieubeheer.

Sinds de ontwikkeling van het model wordt EMAV gezamenlijk beheerd door de Vlaamse Milieumaatschappij (VMM) en de Vlaamse Landmaatschappij (VLM). De VMM staat in voor de wetenschappelijke methodologie en de verdere ontwikkeling van het model, terwijl de VLM de noodzakelijke landbouwkundige gegevens en domeinexpertise aanlevert. Door deze samenwerking blijft EMAV actueel en wetenschappelijk onderbouwd, zodat het model betrouwbare emissieberekeningen kan leveren ter ondersteuning van beleidsvorming, vergunningverlening en wetenschappelijk onderzoek (en Vlaamse Milieumaatschappij, 2022).

EMAV begon als een excel bestand. Terwijl bij het Instituut voor Landbouw-, Visserij- en Voedingsonderzoek (ILVO) meer en meer data wordt verzameld, begon de excel te groot te worden en niet meer onderhoudbaar. Naast de grootte zijn er constant nieuwe wetenschappelijke inzichten die gevonden worden. Hierdoor is een excel bestand niet genoeg meer.

Om dat probleem op te lossen is ILVO overgestapt naar een Windows Forms app, geschreven in C#. Dit is al een oud project en is al meerdere keren aangepast naar

een betere, meer complexe versie.

De onderzoekers zijn vooral gekend in python. Ze hebben een basis van C#, maar het is zeker niet makkelijk voor hun om de code van EMAV te navigeren. Zij vinden problemen, foutjes in de resultaten, maar zij kunnen niet zomaar opzoeken waar het probleem zit. Voorlopig maken ze Tickets aan via devops van Azure. Ze kunnen al veel info doorgeven via die beschrijving, maar het is nog steeds een zoektocht voor de it'ers om het probleem te vinden.

Zoals bij nieuwe features, tijdens het overbrengen van de info, gaat die communicatie niet altijd zo goed. Vele onduidelijkheden vinden hun weg in de lijn van communicatie.

De documentatie wordt onderhouden door de onderzoekers, een word bestand met een paar flowcharts. Zij weten dus niet exact hoe EMAV in elkaar zit. Door die communicatie problemen kan het zijn dat wat de onderzoekers willen beschrijven in de documentatie mogelijks niet is hoe het is geïmplementeerd. Zeker bij het overstappen naar nieuwe versies van het model. Hierdoor is er zeker sprake van Documentatie drift.

4.2. Doel

Het doel is om te zoeken naar een manier om EMAV toegankelijker te maken voor onderzoekers die niet bekend zijn met C#. Naast die toegankelijkheid is de communicatie tussen de onderzoekers en de it'ers ook cruciaal.

Vooraf ook checken als de logica klopt.

5

Conclusie

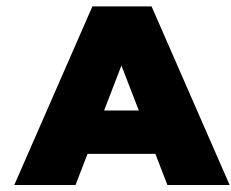
Dit onderzoek richtte zich op de fundamentele vraag hoe de kloof tussen IT-logica en wetenschappelijke analyse binnen de context van ILVO verkleind kan worden. Door de bevindingen en afwegingen uit de opeenvolgende hoofdstukken te synthetiseren, kan een integraal antwoord op de onderzoeksvragen worden geformuleerd.

De uitleg in de *huidige staat van documentatie* leerde dat eerdere pogingen tot documentatie, zoals het bijhouden van “schaduw-code” in losse Markdown-bestanden, faalden door documentation drift. Hoewel deze aanpak voor developers overzichtelijker was dan de lijvige Word-documenten uit het legacy-tijdperk, bleef de informatie gefragmenteerd. De analyse van de *projectcomplexiteit* bevestigde dat een ad-hoc programmeerstijl ontoereikend is voor de hybride datastructuur en legacy-rekenmodules van EMAV. Hoewel alternatieve, eenvoudigere architecturen overwogen zijn, bleek de keuze voor Clean Architecture en een uitgebreid frame van design patterns (zoals Repositories en Strategieën) de enige manier om de wetenschappelijke kern te isoleren. In het hoofdstuk over de *tekortschietsing van documentatie* werd vastgesteld dat zelfs moderne tools zoals DocFX en Swagger, ondanks hun technische kracht, fundamenteel tekortschieten voor onderzoekers omdat zij de intentie en de wiskundige onderbouwing van de modellen niet ontsluiten.

Bij de verkenning van *alternatieve documentatievormen* zijn diverse platforms kritisch tegen elkaar afgewogen. Quarto werd overwogen vanwege het superieure versiebeheer via platte tekst, en Org-mode vanwege de pure implementatie van Literate Programming. Echter, de noodzaak voor een drempelloze interface leidde tot de keuze voor Jupyter-notebooks. Voor de technische koppeling werd Python.NET verkozen boven IronPython; waar IronPython beperkt blijft tot verouderde Python-versies, biedt Python.NET de toegang tot het moderne data-science ecosysteem dat voor onderzoekers essentieel is.

De uitwerking van de *architectuur* en de *Proof-of-Concept* valideerden deze keuzes. Waar directe toegang tot productiedata of het handmatig opzetten van lokale omgevingen te risicovol of complex werd bevonden, boden Docker-sandboxen en lokale data-clonering een veilig alternatief. Het onderzoek bewijst dat een samenwerking van deze technologieën een *Single Source of Truth* realiseert die technisch robuust is en de autonomie van de wetenschapper herstelt.

Concluderend toont dit onderzoek aan dat de hybride integratie van .NET en Jupyter de meest levensvatbare strategie is om de “black-box” van wetenschappelijke software te openen. De belangrijkste beperking is dat een definitieve validatie nog afhankelijk is van een structurele inname van de mening en ervaringen van de onderzoekers zelf. De technische fundering is hiermee gelegd; de menselijke adoptie en verdere automatisatie binnen CI/CD-pipelines vormen de logische vervolgstappen voor de toekomst van EMAV.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Wetenschappelijke instituten zoals ILVO (Instituut voor Landbouw-, Visserij- en Voedingsonderzoek) maken intensief gebruik van complexe rekenmodellen om emissies, productie-impact en andere landbouwparameters te analyseren. Deze modellen bestaan uit clusters van formules die door onderzoekers worden ontwikkeld en vervolgens door de IT-afdeling worden geïmplementeerd in softwaretoepassingen. Aangezien onderzoekers deze toepassingen gebruiken om hypothesen te testen en scenario's te simuleren, is de kwaliteit, transparantie en aanpasbaarheid van deze implementaties van groot belang. In de praktijk blijken dergelijke modellen echter moeilijk wijzigbaar en weinig inzichtelijk zodra ze in code zijn gegoten.

Binnen ILVO wordt dit probleem concreet zichtbaar in .NET-gebaseerde applicaties waarin emissieformules en bedrijfsparameters zijn verwerkt in C#-logica. Onderzoekers beschikken over diepgaande inhoudelijke expertise om hun modellen en aannames te verfijnen, maar hebben niet altijd voldoende programmeerkennis om deze wijzigingen zelfstandig in de software door te voeren. Voor elke aanpassing zijn zij afhankelijk van de IT-afdeling, waardoor zelfs beperkte experimenten of kleine parameterwijzigingen vertraging oplopen. Bovendien is de bestaande documentatie niet altijd nauw verweven met de code, wat het moeilijk maakt om te achterhalen waar specifieke berekeningen plaatsvinden en welke aannames eraan ten grondslag liggen.

De doelgroep van dit onderzoek bestaat uit twee nauw samenwerkende groepen binnen ILVO: enerzijds de onderzoekers die de wetenschappelijke modellen ont-

wikkelen, en anderzijds de IT-professionals die deze modellen implementeren, onderhouden en documenteren. Beide groepen ondervinden hinder van de huidige werkwijze: onderzoekers ervaren een gebrek aan autonomie en inzicht, terwijl IT-professionals disproportioneel veel tijd besteden aan kleine aanpassingen, ondersteuning en foutopsporing.

Vanuit deze concrete probleemsituatie wordt in deze bachelorproef de volgende centrale onderzoeksvraag geformuleerd: *Hoe kunnen we binnen een .NET-gebaseerde (C# & Razor Pages) onderzoeksapplicatie de kloof verkleinen tussen IT-technische logica en wetenschappelijke analyse, zodat onderzoekers van ILVO zonder diepgaande programmeerkennis inzicht krijgen in de werking van hun intern model en er zelf mee kunnen experimenteren?*

Om deze onderzoeksvraag te kunnen beantwoorden, wordt het onderzoek opgesplitst in een aantal samenhangende deelvragen die zowel het probleemdomain als het oplossingsdomain bestrijken. Eerst wordt onderzocht hoe de huidige samenwerking en workflow tussen onderzoekers en IT-professionals verloopt en waar deze in de praktijk vastloopt. Daarnaast wordt nagegaan welke onderdelen van het bestaande .NET-model en de bijhorende code moeilijk te begrijpen of aan te passen zijn voor onderzoekers. Ook wordt onderzocht in welke mate de huidige documentatie tekortschiet in het zichtbaar maken van berekeningen, aannames en afhankelijkheden binnen het model.

Op basis van deze probleemanalyse richt het onderzoek zich vervolgens op mogelijke oplossingsrichtingen. Daarbij wordt onderzocht welke documentatie- en interactievormen onderzoekers ondersteunen bij het begrijpen van wetenschappelijke modellen, zoals inline annotaties, interactieve documentatie of computationele notebooks. Verder wordt nagegaan hoe onderzoekers veilig kunnen experimenteren met modelparameters en formules, bijvoorbeeld via simulaties of gevoeligheidsanalyses, zonder de onderliggende softwarestructuur te doorbreken. Tot slot wordt onderzocht hoe documentatie en code nauwer met elkaar verweven kunnen worden zodat wijzigingen aan het model automatisch transparant blijven voor alle betrokkenen.

Het doel van dit onderzoek is het ontwikkelen van een proof-of-concept waarin documentatie, modelberekeningen en experimenteerruimte voor onderzoekers dicht bij elkaar worden gebracht. Dit proof-of-concept moet aantonen hoe onderzoekers op een gecontroleerde en begrijpelijke manier met hun modellen kunnen werken, terwijl de onderhoudbaarheid en technische kwaliteit van de software behouden blijft. Het uiteindelijke resultaat bestaat uit een concreet voorstel en prototype dat ILVO kan ondersteunen bij het toegankelijker, transparanter en duurzamer maken van hun onderzoeksmodellen.

Dit onderzoek wordt bewust afgebakend: het richt zich niet op het herontwerpen van het volledige ILVO-model of het bouwen van een productieklare applicatie, maar op het ontwikkelen en evalueren van technieken, workflows en tooling die

de interactie tussen code en onderzoek verbeteren. Het theoretisch kader wordt gezocht binnen software engineering, documentatieonderzoek, literate programming, domain-driven design en transparantie van wetenschappelijke modellen.

A.2. Literatuurstudie

A.2.1. Literate programming

Literate programming (Knuth, [g.d.](#)) werd geïntroduceerd door Knuth (1992) als een methode waarbij code en documentatie 1 geïntegreerd geheel vormen, eerder dan gescheiden artefacten. De bedoeling is dat er 1 bestand is met de code en de documentatie errond. De tools halen de code eruit en creëren een uitvoerbaar bestand en een ander bestand met de documentatie, dat kan een html web pagina zijn, of een $\text{\LaTeX}$ bestand / pdf. Het kan ook een Markdown bestand zijn.

In plaats van ‘code with comments’ vertrekt literate programming vanuit het idee dat een programma in de eerste plaats een verhaal is dat aan een menselijke lezer moet worden uitgelegd, waarbij uitvoerbare code een secundaire afgeleide vorm is (Knuth, 1992). Sindsdien zijn er talrijke systemen ontstaan die Knuths principes toepassen in een moderne ontwikkelingsomgeving.

Hoewel Knuths oorspronkelijke WEB en CWEB nog weinig gebruikt worden in de hedendaagse softwarepraktijk, leven zijn ideeën voort in computationele notebooks, literate programminglibraries en research pipelines. Tools zoals Jupyter Notebooks, Org-mode Babel en Quarto implementeren een moderne interpretatie van Knuths ‘weave’ en ‘tangle’-proces, waarin broncode zowel als uitvoerbaar script en als leesbare documentatie dient.

De execution haalt de source code eruit en zorgt voor een uitvoerbaar bestand. De Weave operatie zorgt er voor dat de documentatie uit het source bestand wordt gehaald en die in het gewenste formaat zet. De Tangle operatie verspreidt de source code in de respectievelijke bestanden zelf, niet direct uitvoerbaar.

A.2.2. Org-mode Babel

Org mode is een ‘major mode’ voor GNU Emacs. Org mode kan voor heel veel gebruikt worden: van notities nemen tot todo-lists, computational notebooks en literate programming (Dominik & Schulte, 2025).

In Org Mode met Babel zijn er ‘src-blocks’:

```

1  * Src-blocks in org-mode
2  #+BEGIN_SRC python :results verbatim
3      nummer: int = 10
4      return nummer
5  #+END_SRC
6

```

```
7 #+RESULTS:  
8 : 10
```

Hierin kan code worden uitgevoerd terwijl er tekst rond staat. Dit is het idee van literate programming. De meeste 'cellen' in org-mode hebben hun eigen omgeving, maar er bestaan manieren om dit te vermijden. Babel verbetert de src-blocks door een paar zaken te voorzien:

- interactieve en programma executie van code blokken
- code blokken die functies met parameters hebben, kunnen andere code blokken accepteren en oproepen, en
- bestanden exporteren voor literate programming

A.2.3. Jupyter notebook

Jupyter notebooks zijn gemaakt voor python. Deze notebooks worden door vele mensen gebruikt en in verschillende omgevingen. Dit kan gaan van een student die python leert en zijn notities er bij wil schrijven tot een volledige data analyse door professionele onderzoekers. Dit is een standaard voor computationeel onderzoek en datascience (Project Jupyter, [2025](#)). Het werkt met verschillende 'cellen': de markdown cellen en de python cellen. De python cellen verbinden met een virtuele omgeving en voeren hierop telkens hun code uit. Aangezien ze werken op dezelfde omgeving zijn de cellen verbonden aan elkaar. Het grootste verschil met org-mode is dat een .org bestand een plain tekst bestand is. Het is mogelijk om dit bestand te bekijken in eender welke editor. Jupyter notebooks niet. Je hebt een editor nodig die jupyter notebooks ondersteunt. Het bekendste is VsCode van Microsoft. Er bestaat ook de officiële command en app van jupyter zelf die een server opstart en dan in de browser beschikbaar is.

Maar jupyter notebooks hebben zeker hun minpunten. Rule e.a. ([2018](#)) tonen aan dat Jupyter Notebooks niet-lineaire uitvoer geschiedenis hebben, wat reproduceerbaarheid ernstig bemoeilijkt. Ook omdat de cellen in een willekeurige volgorde kunnen worden uitgevoerd, moet er opgelet worden met de run volgorde. De notebooks hebben ook een onzichtbare global state. Het zijn json bestanden, dus versie controle beheer kan hier ook moeilijk mee doen. Aangezien dat alles ook in 1 bestand zit, kan het snel 'bloated' zijn.

A.2.4. RMarkdown

RMarkdown (RStudioPBC, [2020](#)) is ook een computational notebook zoals Jupyter Notebooks, maar het ondersteunt meerdere talen. Het volgt een structuur die sterk aansluit bij de principes van literate programming. Een .Rmd-bestand wordt verwerkt door knitr (Xie e.a., [2025](#)), een transparante engine voor het genereren van

dynamische rapporten in R, ontwikkeld binnen de softwareomgeving voor statistische computing R (The R Foundation, [g.d.](#)).

Knitr voert alle code blokken uit en maakt een nieuw Markdown bestand aan met alle code en hun uitkomsten. Het gemaakte Markdown bestand wordt dan gevalueerd door pandoc (MacFarlane, [2025](#)), die dan het finale product maakt. Vb:

```

1 ---
2 title: "viridis demo"
3 output: html_document
4 ---
5
6 ```{r include - FALSE}
7 library(viridis)
8 ```
9 The code below ...
10
11 ## Colors
12 ```{r}
13 image(volcano, col = viridis(200))
14 ```

```

A.2.5. Tussenbesluit

A.3. Methodologie

Binnen een concrete casus bij het Instituut voor Landbouw-, Visserij- en Voedingsonderzoek (ILVO) wordt een toegepast, onderwerpgericht onderzoeksoptzet gevolgd. De focus ligt op het analyseren van de huidige samenwerking tussen onderzoekers en softwareontwikkelaars rond een .NET-gebaseerd rekenmodel, en op het uitwerken van een haalbare oplossingsrichting die deze samenwerking ondersteunt. De methodologie is opgebouwd uit vijf opeenvolgende fasen, waarin zowel het probleemdomein als het oplossingsdomein systematisch worden onderzocht.

A.3.1. Eerste fase

In de eerste fase wordt het probleemdomein in kaart gebracht. Hierbij wordt onderzocht hoe onderzoekers en ontwikkelaars momenteel samenwerken rond het bestaande model, waar verantwoordelijkheden liggen en op welke punten de workflows elkaar belemmeren. Deze analyse vertrekt vanuit concrete use-cases, zoals het aanpassen van emissieformules, het testen van alternatieve parameters en het uitvoeren van gevoeligheidsanalyses. De gegevens worden verzameld via documentanalyse en overlegmomenten met betrokkenen, met als doel inzicht krijgen

in waar de kloof tussen wetenschappelijke logica en technische implementatie zich manifesteert.

A.3.2. Tweede fase

De tweede fase richt zich op het identificeren van noden en vereisten van beide doelgroepen. Voor onderzoekers gaat het onder meer om transparantie van berekeningen, experimenteerruimte en begrijpelijke documentatie; voor ontwikkelaars om onderhoudbaarheid, stabiliteit en controle over data en interfaces. Deze noden worden gestructureerd geformuleerd als functionele en niet-functionele requirements, die richtinggevend zijn voor het verdere onderzoek. Hierbij wordt expliciet rekening gehouden met organisatorische aspecten, zoals taakverdeling en communicatiestromen, in lijn met inzichten uit onder meer Conway's Law (Conway, 1968).

A.3.3. Derde fase

In de derde fase wordt het oplossingsdomein onderzocht aan de hand van een gerichte literatuurstudie en analyse van bestaande tools en workflows. Concepten zoals Knuth (g.d.), computational notebooks (bijvoorbeeld jupyter notebook (Project Jupyter, 2025) en quarto (Posit Software, PBC, g.d.)), documentatieplatformen (zoals docFX (.NET Foundation, 2025)) en model life-cycle tools (zoals mlflow (LF AI & Data Foundation, 2025)) worden bestudeerd op hun relevantie voor gelijkaardige samenwerkingsproblemen. Daarbij wordt nagegaan in welke mate deze benaderingen geschikt zijn binnen een .NET-context, en welke principes overdraagbaar zijn, ook wanneer de tools zelf technologisch niet rechtstreeks inzetbaar zijn.

A.3.4. Vierde fase

Op basis van de voorgaande fasen volgt in de vierde fase een selectie en ontwerp van een oplossingsrichting. De eerder geïdentificeerde requirements worden gebruikt om mogelijke oplossingen af te wegen en te beperken tot een haalbare subset. Hierbij wordt bewust gekozen voor een beperkte scope, met focus op het verbeteren van transparantie, documentatie en experimenteermogelijkheden, zonder het bestaande model volledig te herontwerpen. Het resultaat van deze fase is een concreet concept voor een ondersteunende workflow of tool, afgestemd op de ILVO-casus.

A.3.5. Vijfde fase

In de vijfde en laatste fase wordt deze oplossingsrichting geconcretiseerd in een proof-of-concept. Dit prototype demonstreert hoe onderzoekers inzicht kunnen krijgen in modelberekeningen en parameters, en hoe zij op een gecontroleerde manier kunnen experimenteren zonder diepgaande programmeerkennis. Het proof-of-concept wordt geevalueerd aan de hand van de eerder gedefinieerde

use-cases en requirements. De bevindingen uit deze evaluatie vormen de basis voor conclusies en aanbevelingen over toepasbaarheid en meerwaarde van de voorgestelde aanpak binnen ILVO en gelijkaardige onderzoekscontexten.

A.4. Verwacht resultaat, conclusie

A.4.1. Probleem- en domeinanalyse

Een helder overzicht van de *current flow* (onderzoekers → IT → applicatie)

Een lijst van problemen, zoals:

- beperkte transparantie
- onvoldoende verweven documentatie
- moeilijk traceerbaarheid van berekeningen
- lange feedbackloops

A.4.2. Literatuurstudie & technologieverkenning

Een lijst van mogelijke technieken en tools om de kloof tussen code en onderzoek te verkleinen. Ook bepalen welke technieken *haalbaar* en *relevant* zijn voor ILVO.

A.4.3. Proof-of-Concept ontwikkeling

Een geïntegreerde POC die aantoont hoe onderzoekers:

- parameters kunnen wijzigen,
- resultaten kunnen testen,

zonder afhankelijk te zijn van IT voor kleine experimenten.

A.4.4. Evaluatie en validatie

Conclusie over welke aanpak het meest effectief is om de kloof te verkleinen en een concreet advies voor ILVO met mogelijke roadmap.

Bibliografie

- Conway, M. (1968). Commitees paper [Geraadpleegd op 20-01-2026]. *Datamation magazine*.
- Dominik, C. & Schulte, E. (2025, oktober 24). *Org mode for Gnu Emacs* [geraadpleegd op 2025-12-10]. Verkregen 24 oktober 2025, van <https://orgmode.org/index.html>
- en Vlaamse Milieumaatschappij, V. (2022). *Protocol VLM - VMM Emissie Model Ammoniak Vlaanderen (EMAV)* (Protocol). Vlaamse Landmaatschappij (VLM) & Vlaamse Milieumaatschappij (VMM). Verkregen 24 juli 2026, van [https://www.vlm.be/nl/SiteCollectionDocuments/GDPR/20220607%5C%20protocol%5C%20VLM%5C%20-%5C%20VMM%5C%20Emissie%5C%20Model%5C%20Ammoniak%5C%20Vlaanderen%5C%20\(EMAV\).pdf](https://www.vlm.be/nl/SiteCollectionDocuments/GDPR/20220607%5C%20protocol%5C%20VLM%5C%20-%5C%20VMM%5C%20Emissie%5C%20Model%5C%20Ammoniak%5C%20Vlaanderen%5C%20(EMAV).pdf)
- Foqué, D. (2009). *Optimalisering en actualisering van de emissie-inventaris ammoniak landbouw: Rapport en handleiding* (P. Demeyer, Red.). Instituut voor Landbouw-, Visserij- en Voedingsonderzoek.
- Knuth, D. E. (g.d.). *Literate Programming* [Accessed 10 December 2025]. Verkregen 10 december 2025, van <http://www.literateprogramming.com/>
- Knuth, D. E. (1992). *Literate Programming*. Center for the Study of Language; Information.
- LF AI & Data Foundation. (2025). *MLflow: An open source platform for the Machine Learning Lifecycle* [[Open-source software], Geraadpleegd op 2026-01-21]. <https://mlflow.org/>
- MacFarlane, J. (2025). *Pandoc, The universal document converter*. <https://pandoc.org/>
- .NET Foundation. (2025). *DocFX: Documentation Generator for .NET* [[Source Code], Geraadpleegd op 2026-01-21]. <https://github.com/dotnet/docfx>
- Posit Software, PBC. (g.d.). *Official quarto github repository* [geraadpleegd op 2025-12-10]. <https://github.com/quarto-dev/quarto-cli/>
- Project Jupyter. (2025). *Project Jupyter / Home* [geraadpleegd op 2025-12-11]. <https://jupyter.org/>
- RStudioPBC. (2020). *R Markdown* [geraadpleegd op 2025-12-12]. <https://rmarkdown.rstudio.com/>
- Rule, A., Tabard, J. D. & Hollan, J. D. (2018). Exploration and Explanation in Computational Notebooks [geraadpleegd op 2025-12-11]. *Proceedings of the 2018 CHI*

Conference on Human Factors in Computing Systems, 32, 1–12. <https://doi.org/https://doi.org/10.1145/3173574.3173606>

The R Foundation. (g.d.). *R: The Project for Statistical Computing* [Geraadpleegd op 2025-12-12]. <https://www.r-project.org/>

Xie, Y. e.a. (2025). *knitr* (Versie 1.50) [Source code. Geraadpleegd op 12-10-2025]. <https://github.com/yihui/knitr>